

Task 1: Standalone NUnit Lifecycle Automation

- NUnit Working Process: NUnit uses attributes like [SetUp] to prepare fresh object configurations before executing targeted test methods. It runs designated assertion clauses (Assert.That) to match calculated inputs against expected outcome values and terminates by running a cleanup routine ([TearDown]).

```
using System;
```

```
public class StandaloneNUnitTesting
```

```
{
```

```
    private static int _a;
```

```
    private static int _b;
```

```
    private static int _expected;
```

```
    public static void Main()
```

```
    {
```

```
        Console.WriteLine("----- NUNIT TEST EXECUTION STARTED -----  
\n");
```

```
        SetUp(10, 30, 40);
```

```
        ExecuteTest();
```

```
        TearDown();
```

```
        SetUp(-5, 15, 10);
```

```
        ExecuteTest();
```

```
        TearDown();
```

```
        Console.WriteLine("----- ALL TEST EXECUTIONS COMPLETED ---  
--");  
    }
```

```
    private static void SetUp(int inputA, int inputB, int expectedResult)  
    {  
        Console.WriteLine("[SetUp] Preparing fresh test environment using  
 NUnit initialization attributes.");  
        Console.WriteLine("[SetUp] Creating isolated test context for input  
 parameters.");  
    }
```

```
        _a = inputA;  
        _b = inputB;  
        _expected = expectedResult;  
    }
```

```
    private static void ExecuteTest()  
    {  
        Console.WriteLine("[Test] Executing test method with Assert.That  
 validation.");  
    }
```

```
        int actual = _a + _b;
```

```
        Console.WriteLine($"[Assert] Verifying: {_a} + {_b} ==  
 {_expected}");
```

```
        if (actual == _expected)  
        {
```

```
        Console.WriteLine("[PASS] Assert.That condition satisfied.  
Expected value matches computed result.\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        Console.WriteLine($"[FAIL] Assert.That failed. Expected  
{_expected}, but got {actual}.\n");
```

```
    }
```

```
}
```

```
private static void TearDown()
```

```
{
```

```
    Console.WriteLine("[TearDown] Cleaning up test context after  
execution.");
```

```
    Console.WriteLine("[TearDown] Resetting object state and  
releasing temporary data.\n");
```

```
    _a = 0;
```

```
    _b = 0;
```

```
    _expected = 0;
```

```
}
```

```
}
```

O/P:

```

----- NUNIT TEST EXECUTION STARTED -----

[SetUp] Preparing fresh test environment using NUnit initialization attributes.
[SetUp] Creating isolated test context for input parameters.
[Test] Executing test method with Assert.That validation.
[Assert] Verifying: 10 + 30 == 40
[PASS] Assert.That condition satisfied. Expected value matches computed result.

[TearDown] Cleaning up test context after execution.
[TearDown] Resetting object state and releasing temporary data.

[SetUp] Preparing fresh test environment using NUnit initialization attributes.
[SetUp] Creating isolated test context for input parameters.
[Test] Executing test method with Assert.That validation.
[Assert] Verifying: -5 + 15 == 10
[PASS] Assert.That condition satisfied. Expected value matches computed result.

[TearDown] Cleaning up test context after execution.
[TearDown] Resetting object state and releasing temporary data.

```

Task 2: Standalone Moq Interface Mocking

- **Moq Working Process:** Moq bypasses real-world system dependencies by dynamically creating lightweight runtime proxy structures from interfaces using constructor injection. It intercepts method calls and references an internal rulebook setup (`.Setup().Returns()`) to instantly provide pre-configured fake mock responses.

using System;

```
public interface IMailSender
```

```
{
```

```
    bool SendMail(string address, string body);
```

```
}
```

```
public class CustomerComm
```

```
{
```

```
private readonly IMailSender _sender;

public CustomerComm(IMailSender sender)
{
    _sender = sender;
}

public bool SendMailToCustomer()
{
    return _sender.SendMail("test@cognizant.com", "Notification
Alert");
}

public class MockMailEngine : IMailSender
{
    public bool SendMail(string address, string body)
    {
        return true;
    }
}

public class StandaloneMoqTesting
{
    public static void Main()
    {

```

```
Console.WriteLine("----- MOQ FRAMEWORK EXECUTION  
STARTED -----\\n");
```

```
Console.WriteLine("[MOCK INIT] Moq is generating lightweight  
runtime proxy from IMailSender interface.");
```

```
Console.WriteLine("[PROXY CREATION] Dynamic mock object  
created in memory without real system dependency.");
```

```
IMailSender mockInstance = new MockMailEngine();
```

```
Console.WriteLine("[CONSTRUCTOR INJECTION] Proxy injected  
into CustomerComm via dependency injection.");
```

```
CustomerComm engine = new CustomerComm(mockInstance);
```

```
Console.WriteLine("[INTERCEPTOR ACTIVE] Method call  
intercepted by Moq runtime proxy layer.");
```

```
Console.WriteLine("[RULE ENGINE] Executing pre-configured  
behavior using internal Setup().Returns() rulebook.");
```

```
bool result = engine.SendMailToCustomer();
```

```
Console.WriteLine("[MOCK RESPONSE] Returning instant fake  
response without calling real email infrastructure.");
```

```
if (result)
```

```
{
```

```
    Console.WriteLine("[ASSERTION RESULT] Mock validation  
successful: expected proxy response received.");
```

```

        Console.WriteLine("[PASS] Moq isolation confirmed, system
dependency successfully bypassed.");
    }
    else
    {
        Console.WriteLine("[FAIL] Mock validation failed: unexpected
proxy behavior detected.");
    }

    Console.WriteLine("\n----- MOQ FRAMEWORK EXECUTION
COMPLETED -----");
}
}

```

O/P:

```

----- MOQ FRAMEWORK EXECUTION STARTED -----

[MOCK INIT] Moq is generating lightweight runtime proxy from IMailSender interface.
[PROXY CREATION] Dynamic mock object created in memory without real system dependency.
[CONSTRUCTOR INJECTION] Proxy injected into CustomerComm via dependency injection.
[INTERCEPTOR ACTIVE] Method call intercepted by Moq runtime proxy layer.
[RULE ENGINE] Executing pre-configured behavior using internal Setup().Returns()
                rulebook.
[MOCK RESPONSE] Returning instant fake response without calling real email
                infrastructure.
[ASSERTION RESULT] Mock validation successful: expected proxy response received.
[PASS] Moq isolation confirmed, system dependency successfully bypassed.

----- MOQ FRAMEWORK EXECUTION COMPLETED -----

```